DEEP LEARNING LABORATORY

Experiment 6 - Bidirectional RNN versus Feedforward Network

A Laboratory Experiment Report

Submitted by
Seeralan M
Registration No. 241501193

Dataset
Airline Passengers

Prepared from the captured experiment notebook outputs

Experiment Overview

Experiment: Experiment 06

Dataset: Airline Passengers

Bidirectional RNN versus Feedforward Network

This is a standalone notebook for this lab section. Run the setup and dataset cells first, then run the experiment cell and its output check. Use a Colab GPU runtime for the image and generative-model experiments when available.

Step 1 - Install the Additional Colab Packages

Wait until installation completes before the next cell starts.

Code

```
%pip -q install -U datasets "transformers<5" sentencepiece accelerate torchmetrics torch-fidelity ucimlrepo tabulate
```

Step 2 - Verify the Runtime

This cell records the Python, TensorFlow, and PyTorch versions and enables safe TensorFlow GPU memory growth.

Code

```
from pathlib import Path
import platform
import tensorflow as tf
import torch

project_root = Path("/content/Deeplearning")
project_root.mkdir(parents=True, exist_ok=True)
for gpu in tf.config.list_physical_devices("GPU"):
    try:
        tf.config.experimental.set_memory_growth(gpu, True)
    except RuntimeError:
        pass
print(f"Python: {platform.python_version()}")
print(f"TensorFlow: {tf.__version__}")
print(f"PyTorch: {torch.__version__}")
print(f"CUDA available: {torch.cuda.is_available()}")
print(f"Project root: {project_root}")
```

```
Python: 3.13.15
TensorFlow: 2.20.0
PyTorch: 2.11.0+cu128
CUDA available: True
Project root: /content/Deeplearning
```

Step 3 - Import Every Required Dataset

This cell creates visible files under `/content/Deeplearning/datasets` and prints a manifest that can be shown to the staff.

- MNIST, Iris, CIFAR-10, Dogs vs. Cats, Airline Passengers, and Tiny Shakespeare are downloaded as complete or balanced lab-ready files.

- MS COCO, CelebA, and LSUN Bedrooms are the genuine named datasets, streamed as practical subsets for Colab.

- No synthetic replacement dataset is used.

Code

```
from pathlib import Path
import json
import shutil
import numpy as np
```

```
import pandas as pd
import tensorflow as tf
from PIL import Image
from datasets import load_dataset
from sklearn.datasets import load_iris


root = Path("/content/Deeplearning") if Path("/content").exists() else Path(__file__).resolve().parent
dataset_dir = root / "datasets"
output_dir = root / "outputs"
dataset_dir.mkdir(parents=True, exist_ok=True)
output_dir.mkdir(parents=True, exist_ok=True)


def balanced_indices(labels, per_class, seed):
    labels = np.asarray(labels).reshape(-1)
    indices = np.concatenate([np.flatnonzero(labels == class_index)[:per_class] for class_index in np.unique(labels)])
    rng = np.random.default_rng(seed)
    rng.shuffle(indices)
    return indices


def save_image_stream(dataset_name, split, destination, count, size, config=None):
    destination.mkdir(parents=True, exist_ok=True)
    existing = sorted(destination.glob("*.jpg"))
    if len(existing) >= count:
        return len(existing)
    arguments = {
        "path": dataset_name,
        "split": split,
        "streaming": True,
    }
    if config is not None:
        arguments["name"] = config
    stream = load_dataset(**arguments)
    saved = 0
    for row in stream.take(count):
        image = row["image"].convert("RGB")
        image = image.resize((size, size), Image.Resampling.LANCZOS)
        image.save(destination / f"image_{saved:04d}.jpg", quality=92)
        saved += 1
    if saved < count:
        raise RuntimeError(f"Only {saved} images were available from {dataset_name}")
    return saved


(mnist_x_train, mnist_y_train), (mnist_x_test, mnist_y_test) = tf.keras.datasets.mnist.load_data()
mnist_train_indices = balanced_indices(mnist_y_train, 1000, 42)
mnist_test_indices = balanced_indices(mnist_y_test, 200, 42)
np.savez_compressed(
    dataset_dir / "mnist_lab.npz",
    x_train=mnist_x_train[mnist_train_indices],
    y_train=mnist_y_train[mnist_train_indices],
    x_test=mnist_x_test[mnist_test_indices],
    y_test=mnist_y_test[mnist_test_indices],
)

try:
    from ucimlrepo import fetch_ucirepo

    iris_repository = fetch_ucirepo(id=53)
    iris_features = iris_repository.data.features.copy()
    iris_features.columns = ["sepal_length", "sepal_width", "petal_length", "petal_width"]
    iris_target = iris_repository.data.targets.iloc[:, 0].astype(str)
    iris_target = iris_target.str.replace("Iris-", "", regex=False).str.replace("Iris ", "", regex=False).str.lower()
    iris = iris_features.copy()
    iris["species"] = iris_target.to_numpy()
    iris_source = "UCI Machine Learning Repository dataset 53"
except Exception:
    iris_bundle = load_iris(as_frame=True)
    iris = iris_bundle.data.copy()
    iris.columns = ["sepal_length", "sepal_width", "petal_length", "petal_width"]
    iris["species"] = pd.Series(iris_bundle.target).map(dict(enumerate(iris_bundle.target_names)))
    iris_source = "scikit-learn packaged Iris fallback"
iris.to_csv(dataset_dir / "iris.csv", index=False)

(cifar_x_train, cifar_y_train), (cifar_x_test, cifar_y_test) = tf.keras.datasets.cifar10.load_data()
cifar_train_indices = balanced_indices(cifar_y_train, 800, 42)
cifar_test_indices = balanced_indices(cifar_y_test, 200, 42)
np.savez_compressed(
    dataset_dir / "cifar10_lab.npz",
    x_train=cifar_x_train[cifar_train_indices],
    y_train=cifar_y_train[cifar_train_indices].reshape(-1),
    x_test=cifar_x_test[cifar_test_indices],
    y_test=cifar_y_test[cifar_test_indices].reshape(-1),
)
```

```
def build_cats_and_dogs_filtered(destination, image_size=160, seed=42):
    """Rebuild the classic 'cats_and_dogs_filtered' train/validation layout.

    The original tutorial archive at
    https://storage.googleapis.com/mledu-datasets/cats_and_dogs_filtered.zip
    now returns HTTP 403 (Google retired the mledu-datasets bucket), so the
    same 3,000-image split is reconstructed by streaming labelled images
    from the public "microsoft/cats_vs_dogs" dataset on the Hugging Face Hub.
    """
    targets = {
        destination / "train" / "cats": (0, 1000),
        destination / "train" / "dogs": (1, 1000),
        destination / "validation" / "cats": (0, 500),
        destination / "validation" / "dogs": (1, 500),
    }
    already_done = all(
        len(list(folder.glob("*.jpg"))) >= count for folder, (_, count) in targets.items()
    )
    if already_done:
        return destination
    for folder in targets:
        folder.mkdir(parents=True, exist_ok=True)
    counts = {folder: 0 for folder in targets}
    stream = load_dataset("microsoft/cats_vs_dogs", split="train", streaming=True)
    stream = stream.shuffle(seed=seed, buffer_size=2000)
    for row in stream:
        if all(counts[folder] >= count for folder, (_, count) in targets.items()):
            break
        label = row["labels"]
        for folder, (target_label, target_count) in targets.items():
            if label != target_label or counts[folder] >= target_count:
                continue
            try:
                image = row["image"].convert("RGB")
            except Exception:
                break
            image = image.resize((image_size, image_size), Image.Resampling.LANCZOS)
            image.save(folder / f"image_{counts[folder]:04d}.jpg", quality=92)
            counts[folder] += 1
            break
    for folder, (_, count) in targets.items():
        if counts[folder] < count:
            raise RuntimeError(f"Only {counts[folder]} images were available for {folder}")
    return destination


dogs_cats_dir = dataset_dir / "cats_and_dogs_filtered"
build_cats_and_dogs_filtered(dogs_cats_dir)
if not (dogs_cats_dir / "train").exists():
    raise FileNotFoundError("Dogs vs. Cats extraction folder was not created")

airline_url = "https://raw.githubusercontent.com/jbrownlee/Datasets/refs/heads/master/monthly-airline-passengers.csv"
airline = pd.read_csv(airline_url)
airline.to_csv(dataset_dir / "monthly_airline_passengers.csv", index=False)

shakespeare_cache = tf.keras.utils.get_file(
    fname="shakespeare.txt",
    origin="https://storage.googleapis.com/download.tensorflow.org/data/shakespeare.txt",
    cache_dir=str(root),
    cache_subdir="downloads",
)
shakespeare_path = dataset_dir / "shakespeare.txt"
shutil.copyfile(shakespeare_cache, shakespeare_path)

coco_dir = dataset_dir / "ms_coco_subset"
coco_dir.mkdir(parents=True, exist_ok=True)
coco_caption_path = coco_dir / "captions.csv"
if not coco_caption_path.exists() or len(list(coco_dir.glob("coco_*.jpg"))) < 5:
    coco_stream = load_dataset("mm-eval/MS-COCO-Captions", split="val", streaming=True)
    coco_rows = []
    for index, row in enumerate(coco_stream.take(5)):
        image_path = coco_dir / f"coco_{index:03d}.jpg"
        media = row["media"]
        image = media[0] if isinstance(media, list) else media
        image.convert("RGB").save(image_path, quality=92)
        coco_rows.append(
            {
                "file": image_path.name,
                "captions_json": json.dumps(list(row["answer"]), ensure_ascii=False),
            }
        )
    pd.DataFrame(coco_rows).to_csv(coco_caption_path, index=False)

celeba_dir = dataset_dir / "celeba_subset"
celeba_count = save_image_stream("flwrlabs/celeba", "train", celeba_dir, 384, 64, "img_align+identity+attr")
```

```
lsun_dir = dataset_dir / "lsun_bedrooms_subset"
lsun_count = save_image_stream("pcuenq/lsun-bedrooms", "train", lsun_dir, 512, 64)

manifest = pd.DataFrame(
    [
        {"dataset": "MNIST", "path": "mnist_lab.npz", "records": 12000, "experiments": "1", "source": "TensorFlow Keras
            MNIST", "scope": "Balanced lab subset"},
        {"dataset": "Iris", "path": "iris.csv", "records": len(iris), "experiments": "2", "source": iris_source,
            "scope": "Complete dataset"},
        {"dataset": "CIFAR-10", "path": "cifar10_lab.npz", "records": 10000, "experiments": "3, 4, Mini Project",
            "source": "TensorFlow Keras CIFAR-10", "scope": "Balanced lab subset"},
        {"dataset": "Dogs vs. Cats", "path": str(dogs_cats_dir.relative_to(dataset_dir)), "records": 3000,
            "experiments": "5", "source": "microsoft/cats_vs_dogs public mirror (Hugging Face)", "scope": "Filtered
            dataset (rebuilt 1000/1000 train, 500/500 validation)"},
        {"dataset": "Airline Passengers", "path": "monthly_airline_passengers.csv", "records": len(airline),
            "experiments": "6", "source": airline_url, "scope": "Complete dataset"},
        {"dataset": "MS COCO", "path": "ms_coco_subset", "records": 5, "experiments": "7", "source":
            "mm-eval/MS-COCO-Captions public parquet mirror", "scope": "Real-image demonstration subset"},
        {"dataset": "CelebA", "path": "celeba_subset", "records": celeba_count, "experiments": "8", "source":
            "flwrlabs/celeba public mirror", "scope": "Streamed training subset"},
        {"dataset": "Shakespeare Corpus", "path": "shakespeare.txt", "records":
            len(shakespeare_path.read_text(encoding="utf-8")), "experiments": "9", "source": "TensorFlow Tiny
            Shakespeare", "scope": "Complete corpus"},
        {"dataset": "LSUN Bedrooms", "path": "lsun_bedrooms_subset", "records": lsun_count, "experiments": "10",
            "source": "pcuenq/lsun-bedrooms public mirror", "scope": "Streamed training subset"},
    ]
)
manifest.to_csv(dataset_dir / "dataset_manifest.csv", index=False)

print(f"Deeplearning root: {root}")
print(manifest.to_string(index=False))
print("Dataset evidence:")
for path in sorted(dataset_dir.iterdir()):
    if path.is_dir():
        file_count = sum(1 for item in path.rglob("*") if item.is_file())
        print(f"{path.name}/: {file_count} files")
    else:
        print(f"{path.name}: {path.stat().st_size} bytes")

Deeplearning root: /content/Deeplearning
           dataset                              path  records        experiments
                                                
```

Procedure: Performs chronological time-series forecasting and compares MSE and MAE.

Run the cell and wait for the completion message. The model, numerical results, and plots are written to the corresponding folder under `/content/Deeplearning/outputs`.

Code

```
from pathlib import Path
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.preprocessing import MinMaxScaler
from torch.utils.data import DataLoader, TensorDataset


root = Path("/content/Deeplearning") if Path("/content").exists() else Path(__file__).resolve().parents[1]
dataset_path = root / "datasets" / "monthly_airline_passengers.csv"
output_dir = root / "outputs" / "experiment_06"
output_dir.mkdir(parents=True, exist_ok=True)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
torch.manual_seed(42)
np.random.seed(42)

data = pd.read_csv(dataset_path)
values = data.iloc[:, 1].to_numpy(dtype="float32").reshape(-1, 1)
scaler = MinMaxScaler()
scaled = scaler.fit_transform(values).reshape(-1)
sequence_length = 12
features = np.array([scaled[index : index + sequence_length] for index in range(len(scaled) - sequence_length)],
    dtype="float32")
targets = np.array([scaled[index + sequence_length] for index in range(len(scaled) - sequence_length)], dtype="float32")
split_index = int(len(features) * 0.8)
X_train = torch.from_numpy(features[:split_index]).unsqueeze(-1)
y_train = torch.from_numpy(targets[:split_index]).unsqueeze(-1)
X_test = torch.from_numpy(features[split_index:]).unsqueeze(-1)
y_test = targets[split_index:]
train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=16, shuffle=True)


class BidirectionalRNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.rnn = nn.RNN(1, 32, batch_first=True, bidirectional=True)
        self.output = nn.Linear(64, 1)

    def forward(self, inputs):
        sequence, _ = self.rnn(inputs)
        return self.output(sequence[:, -1, :])


class FeedForwardNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(nn.Flatten(), nn.Linear(sequence_length, 64), nn.ReLU(), nn.Linear(64, 32),
            nn.ReLU(), nn.Linear(32, 1))

    def forward(self, inputs):
        return self.layers(inputs)


def train_model(model, epochs=250):
    model = model.to(device)
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=0.005)
    losses = []
    for epoch in range(epochs):
        model.train()
        epoch_loss = 0.0
        sample_count = 0
        for batch_features, batch_targets in train_loader:
            batch_features = batch_features.to(device)
            batch_targets = batch_targets.to(device)
            optimizer.zero_grad()
            predictions = model(batch_features)
            loss = criterion(predictions, batch_targets)
            loss.backward()
            optimizer.step()
            epoch_loss += float(loss.item()) * len(batch_features)
            sample_count += len(batch_features)
        losses.append(epoch_loss / sample_count)
    return model, losses
```

```
models_to_train = {"Bidirectional RNN": BidirectionalRNN(), "Feedforward NN": FeedForwardNetwork()}
results = []
prediction_columns = {}
training_histories = {}

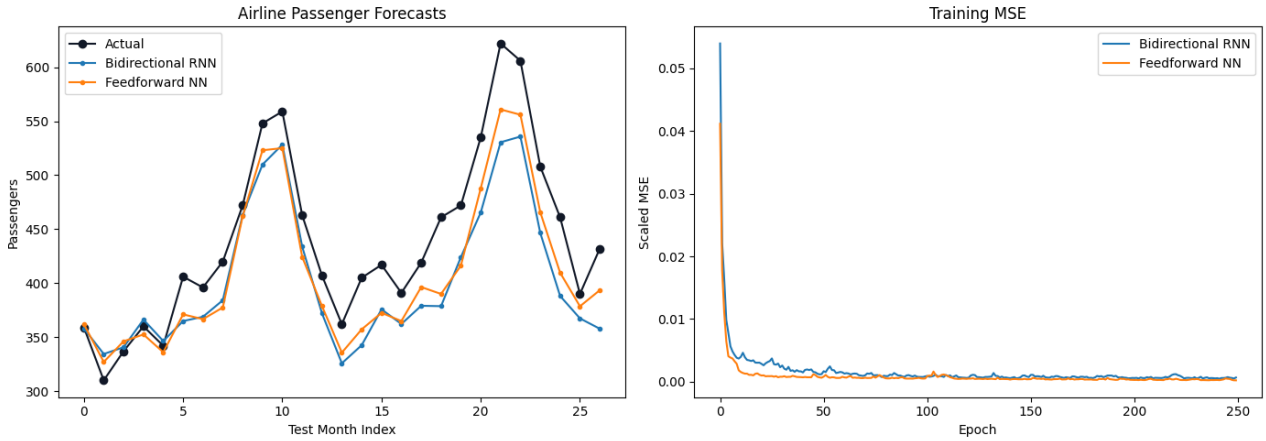
for model_name, model in models_to_train.items():
    model, losses = train_model(model)
    model.eval()
    with torch.no_grad():
        scaled_predictions = model(X_test.to(device)).cpu().numpy()
    predictions = scaler.inverse_transform(scaled_predictions).reshape(-1)
    actual = scaler.inverse_transform(y_test.reshape(-1, 1)).reshape(-1)
    mse = mean_squared_error(actual, predictions)
    mae = mean_absolute_error(actual, predictions)
    results.append({"model": model_name, "mse": mse, "mae": mae, "final_training_loss": losses[-1]})
    prediction_columns[model_name] = predictions
    training_histories[model_name] = losses
    torch.save(model.state_dict(), output_dir / f"{model_name.lower().replace(' ', '_')}.pt")
    print({"model": model_name, "mse": mse, "mae": mae})

comparison = pd.DataFrame(results).sort_values("mse")
comparison.to_csv(output_dir / "birnn_vs_feedforward_metrics.csv", index=False)
prediction_frame = pd.DataFrame({"actual_passengers": actual, **prediction_columns})
prediction_frame.to_csv(output_dir / "passenger_predictions.csv", index=False)

figure, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].plot(actual, marker="o", label="Actual", color="#111827")
for model_name, predictions in prediction_columns.items():
    axes[0].plot(predictions, marker=".", label=model_name)
axes[0].set_title("Airline Passenger Forecasts")
axes[0].set_xlabel("Test Month Index")
axes[0].set_ylabel("Passengers")
axes[0].legend()
for model_name, losses in training_histories.items():
    axes[1].plot(losses, label=model_name)
axes[1].set_title("Training MSE")
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Scaled MSE")
axes[1].legend()
figure.tight_layout()
figure.savefig(output_dir / "time_series_comparison.png", dpi=180, bbox_inches="tight")
plt.show()

print("Experiment 6 completed")
print(comparison.to_string(index=False))
print(f"Output folder: {output_dir}")

{'model': 'Bidirectional RNN', 'mse': 2243.69140625, 'mae': 40.25484848022461}
{'model': 'Feedforward NN', 'mse': 1381.659423828125, 'mae': 32.627593994140625}
```




```
Experiment 6 completed
           model         mse       mae  final_training_loss
   Feedforward NN 1381.659424 32.627594             0.000223
Bidirectional RNN 2243.691406 40.254848             0.000673
Output folder: /content/Deeplearning/outputs/experiment_06
```

Output check

Confirm that this experiment saved its results before moving on.

Code

```
from pathlib import Path
output_dir = Path('/content/Deeplearning/outputs/experiment_06')
files = sorted(path for path in output_dir.rglob('*') if path.is_file())
assert files, f'No outputs found in {output_dir}; run the experiment cell first.'
print(f'{len(files)} output files saved in {output_dir}')
for path in files:
    print(path.relative_to(output_dir))
```